Instituto Federal do Maranhão
Campus Grajaú

Tecnologia em Análise e Desenvolvimento de Sistemas

Lista de Exercícios

Aula 04 — Abstração, Relacionamentos entre Objetos e UML Básica

Disciplina	Programação Orientada a Objetos
Professor	Welington Galvão Rodrigues
Turma	ADS 2026.2
Data	26/08/2026 (Quarta-feira)
Horário	14h40 – 18h15
Valor total	10,0 pontos
Aluno(a)	_____
Matrícula	_____

Instruções Gerais

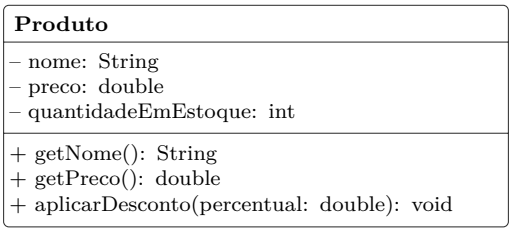
- Cada questão apresenta um diagrama de classes UML. Implemente, em Java, as classes correspondentes, respeitando fielmente os atributos, a visibilidade, os métodos (nomes, parâmetros e tipos de retorno) e, quando indicado, o tipo de relacionamento entre as classes.
- Losango **vazado** no lado do "todo" indica **agregação**; losango **preenchido** indica **composição**.
- É permitido o uso de anotações e material de aula.
- Esta lista compõe a nota de atividades da 1ª Etapa.
- Entregar ao final da aula ou conforme orientação do professor.

Bloco	Conteúdo	Questões	Valor
A	UML Básica	01 – 02	2,0 pts
B	Agregação	03 – 06	4,0 pts
C	Composição	07 – 10	4,0 pts
Total			**10,0 pts**

(2 questões × 1,00 pt = 2,0 pts)

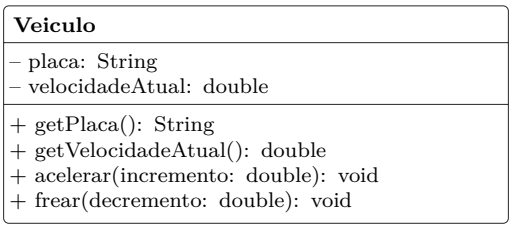
Questão 01 (1,00 pt)

Implemente a classe `Produto` a partir do diagrama abaixo. O método `aplicarDesconto(double percentual)` deve reduzir o `preco` proporcionalmente ao percentual informado (ex.: `aplicarDesconto(10)` reduz o preço em 10%).



Questão 02 (1,00 pt)

Implemente a classe `Veiculo` a partir do diagrama abaixo. O método `frear` não pode deixar `velocidadeAtual` ficar negativa — se o decremento informado for maior que a velocidade atual, a velocidade deve ir para `0`.



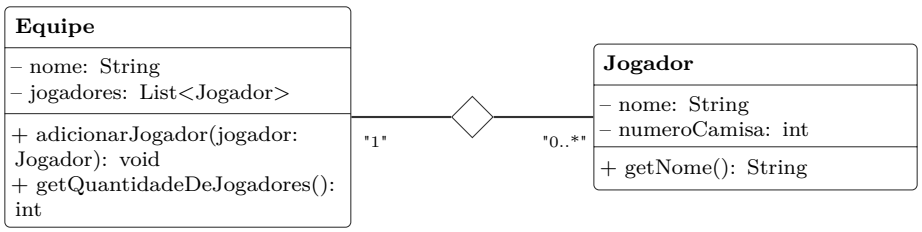
(4 questões × 1,00 pt = 4,0 pts)

▷ **Dica**

Em todas as questões deste bloco, a "parte" deve poder ser criada **independentemente** do "todo" e só depois ser associada a ele — é isso que caracteriza a agregação (losango **vazado**).

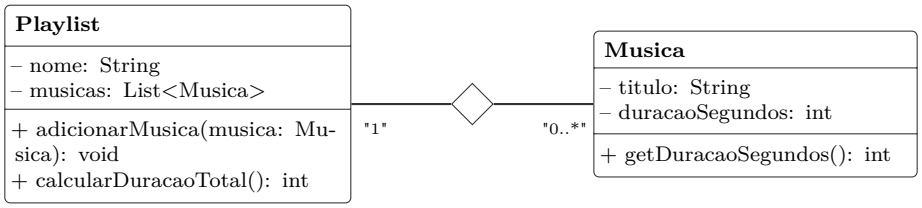
Questão 03 (1,00 pt)

Implemente as classes `Equipe` e `Jogador`. Note que `adicionarJogador` recebe um `Jogador` **já existente** como parâmetro — um `Jogador` deve poder ser instanciado de forma totalmente independente de qualquer `Equipe` antes de ser adicionado.



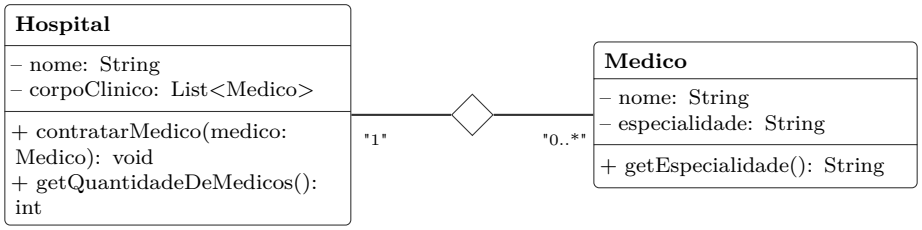
Questão 04(1,00 pt)

Implemente as classes `Playlist` e `Musica`. Como é agregação, a **mesma** instância de `Musica` deve poder ser adicionada a mais de uma `Playlist`. O método `calcularDuracaoTotal()` deve retornar a soma de `duracaoSegundos` de todas as músicas.



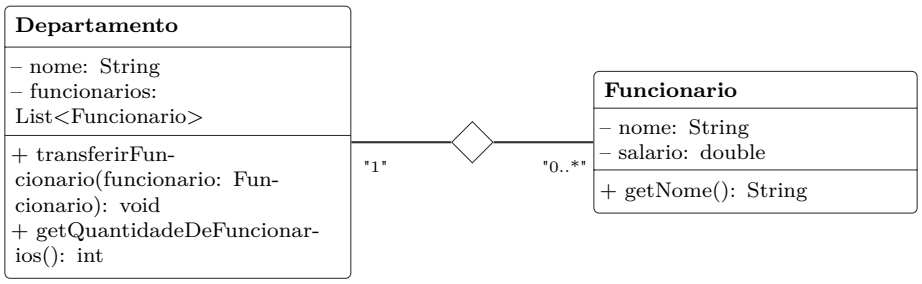
Questão 05 (1,00 pt)

Implemente as classes `Hospital` e `Medico`. Um `Medico` deve poder ser instanciado independentemente e só depois "contratado" por um `Hospital` através de `contratarMedico`.



Questão 06 (1,00 pt)

Implemente as classes `Departamento` e `Funcionario`. O método `transferirFuncionario` deve adicionar um `Funcionario` já existente à lista do departamento (o `Funcionario` continua existindo no sistema independentemente de a qual departamento pertence).



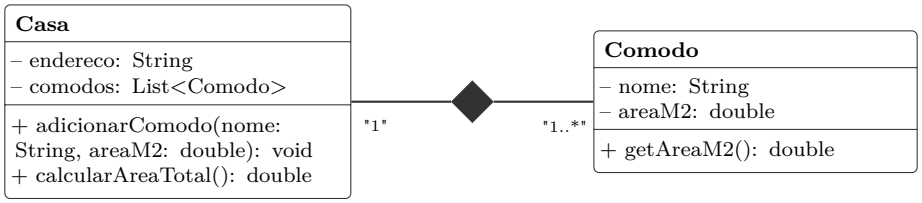
(4 questões × 1,00 pt = 4,0 pts)

▷ **Dica**

Em todas as questões deste bloco, a "parte" **não deve existir fora do todo** — é o próprio "todo" quem deve criar as instâncias da "parte" internamente, recebendo apenas os dados soltos necessários (losango **preenchido**).

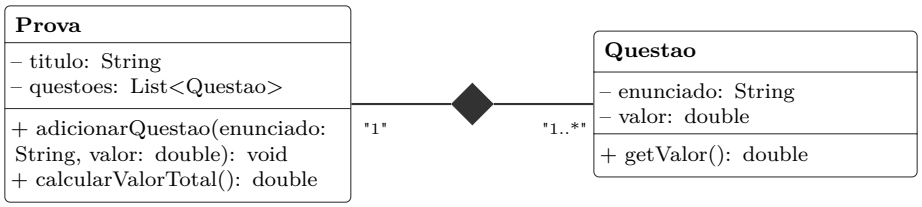
Questão 07 (1,00 pt)

Implemente as classes `Casa` e `Comodo`, respeitando a composição: `adicionarComodo` recebe os dados soltos (`nome`, `areaM2`) e deve criar o `Comodo` internamente — a classe `Casa` nunca recebe um objeto `Comodo` já pronto de fora. `calcularAreaTotal()` deve somar a área de todos os cômodos.



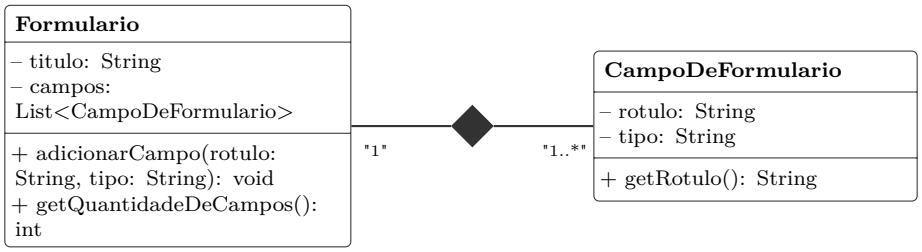
Questão 08(1,00 pt)

Implemente as classes `Prova` e `Questao`, seguindo o mesmo princípio de composição da Questão 07 (a `Prova` cria suas `Questao` internamente). `calcularValorTotal()` deve somar o `valor` de todas as questões.



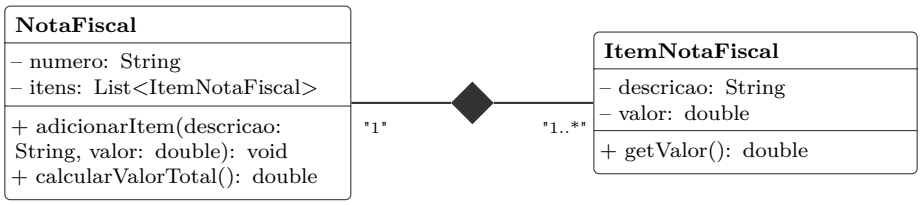
Questão 09(1,00 pt)

Implemente as classes `Formulario` e `CampoDeFormulario`, respeitando a composição.



Questão 10(1,00 pt)

O diagrama abaixo mostra uma relação de **composição** entre `NotaFiscal` e `ItemNotaFiscal`:



O código abaixo **tenta** implementar esse diagrama, mas não reflete corretamente o tipo de relacionamento mostrado:

```java
public class NotaFiscal {
    private String numero;
    private List<ItemNotaFiscal> itens = new ArrayList
       <>();

    public void adicionarItem(ItemNotaFiscal item) {
        itens.add(item);
    }
}
```

(a) Por que esse código **não** reflete corretamente a composição mostrada no diagrama? Qual relacionamento ele estaria implementando, na verdade?

(b) Reescreva a classe `NotaFiscal` (incluindo a assinatura de `adicionarItem`) para refletir corretamente a composição indicada no diagrama.

"Abstrair é escolher o que importa; relacionar objetos é modelar o mundo como ele realmente funciona."

Boa sorte! — Prof. Welington Galvão Rodrigues